Cloud Audit Logs

A Practical Handbook for Google Cloud Governance, Security & Compliance

Covers: Admin Activity, Data Access, System Event & Policy Denied logs • Log structure & the AuditLog object • Log Router, sinks & buckets • IAM roles • A full hands-on demo you can run yourself (console, gcloud CLI, and a companion notebook)

Prepared for GCP Data Engineering Training

Reflects Google Cloud documentation as of July 2026

Table of Contents

1. What Are Cloud Audit Logs?

Cloud Audit Logs is a Google Cloud service (part of Cloud Logging / Cloud Observability) that automatically records **"who did what, where, and when"** across almost every Google Cloud service in your project, folder, or organization. Every time a principal (a user, a service account, or Google itself) calls a Google Cloud API — creates a VM, changes an IAM policy, reads a row from BigQuery, deletes a Cloud Storage bucket — Google Cloud can write a permanent, tamper-evident record of that call. That record is an **audit log entry**.

Unlike application logs that you write yourself, audit logs are generated by Google Cloud's infrastructure itself, so they are trustworthy evidence of what actually happened, not just what your application believes happened. This makes them the backbone of three things every organization eventually needs:

- **Security investigation** — "who deleted this BigQuery dataset at 2 AM, and from which IP address?"
- **Compliance & audit** — proving to an auditor (SOC 2, ISO 27001, RBI/SEBI guidelines, etc.) that access to sensitive data is tracked and reviewable.
- **Operational troubleshooting** — "why did this Cloud Run deployment fail, and who changed the IAM binding that broke it?"

How this fits together with other GCP governance tools

If you've already gone through the companion handbook on **Data Catalog / Knowledge Catalog and policy tags**, think of the relationship this way: policy tags control *whether* someone can read a column; audit logs record *that* they tried, and whether it succeeded or was denied. Governance is usually "prevent + detect" together — policy tags are the prevention layer, audit logs are the detection and evidence layer.

> **NOTE:** Audit logs are **on by default** for the most important category (Admin Activity) and cannot be turned off. Other categories (Data Access, in particular) are off by default because they can be high-volume, and you opt in to them per service. This distinction matters a lot and is covered in Section 3.

2. Core Concepts & Glossary

Read this section before the demo — every step later reuses these exact terms. Definitions are written the way you'd explain them to someone seeing audit logs for the first time.

Term	Plain-English Definition
Audit Log	A record generated automatically by Google Cloud whenever an API call or system action happens on a resource. Distinct from ordinary application/platform logs, which you or a service choose to emit yourself.
Log Entry	The individual, self-contained unit stored for one event. Represented by a LogEntry object with fields like timestamp, severity, resource, and a payload. Every audit log you see in the console is one LogEntry.
AuditLog object / protoPayload	The structured payload inside a LogEntry that carries the actual audit details — who made the call (authenticationInfo), what method was called (methodName), on which resource (resourceName), what the request/response looked like, and whether it was authorized. Found under the protoPayload field of the LogEntry.
principalEmail	The identity (user email, service account email, or Google system) that performed the logged action. This is usually the very first thing you search for during an investigation.
methodName	The specific API method that was called, e.g. google.iam.admin.v1.SetIamPolicy or storage.buckets.delete — tells you exactly what action happened.
resourceName	The full path identifying which resource the action was performed on, e.g. projects/my-project/buckets/my-bucket.
Log Name	The identifier that tells you which of the four audit log types (and which resource container) an entry belongs to, in the form projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2FLOG_TYPE, where LOG_TYPE is activity, data_access, system_event, or policy.
Admin Activity audit logs	Log entries for API calls that modify the configuration or metadata of a resource (e.g. creating a VM, changing an IAM policy). Always on, free, and cannot be disabled.
Data Access audit logs	Log entries for calls that read configuration/metadata (ADMIN_READ), or that read or write user-provided data (DATA_READ / DATA_WRITE) — e.g. running a SELECT query in BigQuery, downloading an object from Cloud Storage. Off by default (except BigQuery, where DATA_READ/DATA_WRITE are always on); must be explicitly enabled per service; billed like other Cloud Logging ingestion once enabled.
System Event audit logs	Log entries for actions Google Cloud systems take on their own, not triggered by a specific human or service-account request — e.g. Compute Engine live-migrating a VM off failing hardware. Always on and free, like Admin Activity.

Term	Plain-English Definition
Policy Denied audit logs	Log entries generated when a request is denied because it violates a security policy (e.g. an Organization Policy or VPC Service Controls perimeter) rather than a normal IAM permission check. Useful for spotting misconfigurations and blocked attack attempts.
ADMIN_READ / ADMIN_WRITE / DATA_READ / DATA_WRITE	The four IAM 'permission types' that Google Cloud tags each permission with internally. ADMIN_WRITE always produces Admin Activity logs (can't disable). ADMIN_READ, DATA_READ, and DATA_WRITE generally produce Data Access logs and are usually opt-in.
Log Router / Sink	The Cloud Logging component that decides, for every incoming log entry, which log bucket(s) or external destination(s) (BigQuery dataset, Cloud Storage bucket, Pub/Sub topic) it gets routed to. A sink is a named rule ('route logs matching this filter to this destination').
Log Bucket	The storage container that actually holds log entries inside Cloud Logging. Every project gets two built-in buckets: _Required (holds Admin Activity, System Event, Access Transparency logs; cannot be modified or deleted) and _Default (holds everything else, including Data Access and Policy Denied logs by default, with configurable retention).
Retention period	How long a log bucket keeps entries before automatically deleting them. _Required bucket retention is fixed (400 days for Admin Activity / System Event logs). _Default bucket retention is configurable from 1 to 3,650 days.
Logs Explorer	The Cloud Console UI for searching, filtering, and viewing log entries interactively using a query language (Logging query language, similar to a structured search syntax).
Log-based Metric	A custom metric derived by counting or extracting values from matching log entries — lets you build alerts (e.g. 'alert if more than 5 IAM policy changes happen in 10 minutes') on top of audit log activity.
Access Transparency	A separate, premium feature that logs actions taken by Google's own staff/support engineers when they access your content (e.g. during a support ticket) — distinct from Cloud Audit Logs, which cover your own users and service accounts.
Exclusion filter	A rule on a sink (or on the router generally) that prevents certain matching logs from being stored or routed, often used to keep noisy, high-volume Data Access logs from blowing past your budget or quota.

3. The Four Types of Audit Logs, Explained

Every audit log entry belongs to exactly one of four types. Knowing which is which — and which cost money — is the single most useful piece of practical knowledge in this handbook.

Type	Default state	Cost	What it captures
Admin Activity	Always on — cannot disable	Free	Config/metadata changes: creating a VM, changing an IAM policy, updating a firewall rule.
Data Access	Off by default for most services (on by default for BigQuery)	Billed like other log ingestion once enabled	Reads of config/metadata (ADMIN_READ) and reads/writes of user data (DATA_READ / DATA_WRITE): a SELECT query in BigQuery, downloading a Cloud Storage object.
System Event	Always on — cannot disable	Free	Google-system-initiated changes not driven by a user request: VM live migration, automatic scaling actions.
Policy Denied	Always on for supported services	Billed like Data Access	Requests blocked by a security policy (Org Policy, VPC Service Controls) rather than a normal permission check.

Why Data Access logs are opt-in

Data Access logs can be extremely high-volume — imagine logging every single row-read across a busy production database. Google Cloud makes you opt in per-service (and even per permission-type) so you only pay for and store what you actually need for compliance or investigation, rather than being flooded by noise. BigQuery is the one major exception: its DATA_READ and DATA_WRITE audit logs are enabled by default because query-level auditing is considered essential for a data warehouse.

> **NOTE:** This handbook's demo focuses on **Data Access logs for BigQuery** because they are the most directly useful pairing with the earlier Data Catalog / policy tags handbook — you can literally see, in the logs, when a user's query was blocked by a policy tag.

4. Anatomy of a Log Entry

A raw audit log entry looks intimidating the first time you see it as JSON. Below is a trimmed, realistic example (a BigQuery query event) with the fields that matter most for a beginner highlighted.


```
{
  "logName": "projects/himanshugcpproject/logs/cloudaudit.googleapis.com%2Fdata_acces
s",
  "resource": {
    "type": "bigquery_project",
    "labels": {"project_id": "himanshugcpproject"}
  },
  "timestamp": "2026-07-16T09:41:02.113Z",
  "severity": "NOTICE",
  "protoPayload": {
    "@type": "type.googleapis.com/google.cloud.audit.AuditLog",
    "authenticationInfo": {
      "principalEmail": "[email protected]"
    },
    "methodName": "google.cloud.bigquery.v2.JobService.Query",
    "resourceName": "projects/himanshugcpproject/jobs/bquxjob_1234",
    "requestMetadata": {
      "callerIp": "203.0.113.42"
    },
    "status": {},
    "serviceData": {
      "jobCompletedEvent": {
        "job": {"jobConfiguration": {"query": {"query": "SELECT * FROM hr_demo.employ
ees"}}}
      }
    }
  }
}
```


Field-by-field, in plain English

Field	Meaning
logName	Which of the four log types this is (here: data_access) and which project it belongs to.
resource.type / resource.labels	What kind of Google Cloud resource generated this event, and its identifying labels.
timestamp	When the event happened, in UTC.
severity	Log severity (NOTICE, ERROR, etc.) — a permission-denied event typically shows as ERROR/PERMISSION_DENIED in protoPayload.status.
protoPayload.authenticationInfo.principalEmail	WHO performed the action — the most-searched field during investigations.
protoPayload.methodName	WHAT action/API method was invoked.
protoPayload.resourceName	WHICH specific resource was acted on.

Field	Meaning
protoPayload.requestMetadata.callerIp	WHERE the request came from (source IP).
protoPayload.status	Empty/absent on success; contains a code and message if the call failed or was denied.
protoPayload.serviceData / metadata	Service-specific extra detail — for BigQuery, this can include the actual SQL query text (useful, but also sensitive — see best practices).

5. IAM Roles You Need to Know

Role (display name)	Role ID	What it lets you do
Logs Viewer	roles/logging.viewer	Read-only access to Admin Activity, System Event, and Policy Denied logs. Does NOT include Data Access logs stored in the _Default bucket.
Private Logs Viewer	roles/logging.privateLogViewer	Everything Logs Viewer has, plus the ability to read Data Access and Policy Denied logs in the _Default bucket. This is the role you actually need to see BigQuery query-level audit logs.
Logs Configuration Writer	roles/logging.configWriter	Create and manage sinks, log buckets, and exclusion filters — needed to route audit logs to BigQuery/Pub/Sub/Storage or change retention.
Logs Bucket Writer / Owner	roles/logging.bucketWriter, roles/logging.bucketOwner	Manage a specific log bucket's configuration and retention.
Logging Admin	roles/logging.admin	Full control over all Cloud Logging resources in a project — typically reserved for platform/SRE teams.
IAM Admin Log Viewer (org-level context)	roles/logging.viewer (granted at org/folder level)	The same viewer role, but granted higher up the resource hierarchy, so you can review activity across an entire folder or organization rather than one project at a time.

Rule of thumb for the demo: give yourself (the trainer/admin) **Private Logs Viewer** + **Logs Configuration Writer**. Give a 'security reviewer' test account only **Private Logs Viewer**, so students can see that viewing audit logs is itself a permission that must be deliberately granted — not something every project member gets by default.

6. Hands-On Demo — Step by Step

This demo enables Data Access audit logs for BigQuery, generates a few real events (a successful query, a failed/denied query, and an IAM change), then finds and reads those events back using both the Logs Explorer console and the gcloud CLI. If you completed the Data Catalog / policy tags handbook first, Step 3 below reuses that same `hr_demo.employees` table so you can literally watch a policy-tag denial show up as an audit log entry.

> **NOTE:** Time required: ~20–30 minutes.

Prerequisites

- A Google Cloud project (e.g. `himanshugcpproject`) with billing enabled.
- Cloud Shell or a local machine with `gcloud` and `bq` authenticated.
- Your account should have **Private Logs Viewer** and **Logs Configuration Writer** (Project Owner covers both).
- (Optional) The `hr_demo.employees` table from the Data Catalog handbook — or any BigQuery table will do.

Set shell variables (used throughout)

```
export PROJECT_ID="himanshugcpproject"
gcloud config set project $PROJECT_ID
```

STEP 1 — Confirm Admin Activity logs (already on) and view one

Admin Activity logs need no setup — trigger one right now by making any config change, e.g. creating a BigQuery dataset:

```
bq mk --location=us --dataset ${PROJECT_ID}:audit_demo

# View it (Console): Logging → Logs Explorer → in the query box:
resource.type="bigquery_dataset"
logName="projects/${PROJECT_ID}/logs/cloudaudit.googleapis.com%2Factivity"

# View it (CLI):
gcloud logging read \
  'logName="projects/'${PROJECT_ID}'/logs/cloudaudit.googleapis.com%2Factivity"' \
  --limit=5 --format=json --freshness=10m
```

STEP 2 — Enable Data Access audit logs for BigQuery

BigQuery's DATA_READ/DATA_WRITE are actually on by default, but ADMIN_READ (reads of table/dataset metadata) is not — let's enable all three explicitly so nothing is missed.

Console: IAM & Admin → Audit Logs → find 'BigQuery' in the service list → check Admin Read, Data Read, Data Write → Save.

Using gcloud (via an IAM policy patch):

```
gcloud projects get-iam-policy $PROJECT_ID --format=json > policy.json

# Edit policy.json and add/merge this block at the top level:
"auditConfigs": [
  {
    "service": "bigquery.googleapis.com",
    "auditLogConfigs": [
      {"logType": "ADMIN_READ"},
      {"logType": "DATA_READ"},
      {"logType": "DATA_WRITE"}
    ]
  }
]

gcloud projects set-iam-policy $PROJECT_ID policy.json
```

STEP 3 — Generate some real events to find later

```
# A successful, allowed query
bq query --use_legacy_sql=false \
"SELECT employee_id, full_name FROM \`${PROJECT_ID}.hr_demo.employees\` LIMIT 10"

# An IAM change (Admin Activity event)
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:[email protected]" \
  --role="roles/bigquery.dataViewer"
```

If you completed the policy tags handbook and still have a restricted test principal, sign in as them and run a query against a tagged column (e.g. `salary`) to generate a **denied** Data Access event — this is genuinely useful to see in the logs.

STEP 4 — Find the events in the Logs Explorer (console)

Logging → Logs Explorer. In the query box, filter by resource type and log type, then narrow by principal or method:

```
resource.type="bigquery_project"
logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Fdata_access"
protoPayload.authenticationInfo.principalEmail="[email protected]"

-- To find only DENIED events:
protoPayload.status.code!=0
```

Click any entry to expand it — you'll see exactly the field structure covered in Section 4. Use the **Actions** menu on the query bar to save this as a starred query for future reuse.

STEP 5 — Query the same events with gcloud (great for scripting)

```
gcloud logging read \
  'logName="projects/'${PROJECT_ID}'/logs/cloudaudit.googleapis.com%2Fdata_access" '\
'AND protoPayload.authenticationInfo.principalEmail="[email protected]"' \
  --limit=20 --format=json --freshness=1h
```

Pipe the JSON output into `jq` or Python to pull out just the fields you care about — e.g. `jq '.[].protoPayload.methodName'`.

STEP 6 — Build a log-based metric + alert (optional but recommended)

Turn 'someone changed an IAM policy' into an alert your team actually sees:

```
gcloud logging metrics create iam_policy_changes \
  --description="Counts IAM policy changes" \
  --log-filter='protoPayload.methodName="SetIamPolicy"'
```

Console: Monitoring → Alerting → Create Policy → choose the `logging/user/iam_policy_changes` metric → set a threshold (e.g. ≥ 1 in 5 minutes) → add a notification channel (email/Slack/PagerDuty).

7. Bonus: Exporting Logs to BigQuery for SQL Analysis

The default 30–400 day retention in Cloud Logging is fine for recent investigation, but for long-term compliance retention and rich SQL analysis (e.g. 'show me every DELETE across all projects last quarter'), export audit logs continuously to a BigQuery dataset using a **sink**.

STEP 7a — Create a BigQuery dataset to receive logs

```
bq mk --location=us --dataset ${PROJECT_ID}:audit_logs_export
```

STEP 7b — Create the sink

```
gcloud logging sinks create audit-logs-to-bq \
  bigquery.googleapis.com/projects/${PROJECT_ID}/datasets/audit_logs_export \
  --log-filter='logName:"cloudaudit.googleapis.com"' \
  --use-partitioned-tables
```

This command prints a service account (something like `[email protected]`). You must grant it **BigQuery Data Editor** on the destination dataset, or the sink will silently fail to write:

```
bq add-iam-policy-binding \
  --member="serviceAccount:PASTE_SINK_SERVICE_ACCOUNT_HERE" \
  --role="roles/bigquery.dataEditor" \
  ${PROJECT_ID}:audit_logs_export
```

STEP 7c — Query the exported logs with SQL

```
SELECT
  timestamp,
  protopayload_auditlog.authenticationInfo.principalEmail AS who,
  protopayload_auditlog.methodName AS action,
  protopayload_auditlog.resourceName AS resource,
  protopayload_auditlog.status.code AS status_code
FROM `PROJECT_ID.audit_logs_export.cloudaudit_googleapis_com_data_access_*`
WHERE _TABLE_SUFFIX BETWEEN '20260701' AND '20260716'
  AND protopayload_auditlog.status.code IS NOT NULL
ORDER BY timestamp DESC
LIMIT 100;
```

NOTE: Exported log tables are date-sharded (one table per day, suffixed YYYYMMDD) when you use `--use-partitioned-tables`, which makes date-ranged queries cheap. This is a great live example for students of _TABLE_SUFFIX wildcard queries.

8. Troubleshooting Common Issues

Symptom	Likely Cause / Fix
“I can't see any Data Access logs in Logs Explorer”	Two likely causes: (1) you only have roles/logging.viewer, not Private Logs Viewer — Data Access logs in the _Default bucket require the private role; (2) Data Access logging isn't enabled for that service yet (check IAM & Admin → Audit Logs).
Sink created but the BigQuery destination dataset stays empty	The sink's auto-generated service account almost always needs BigQuery Data Editor granted explicitly on the destination dataset — this step is easy to miss.
Too many Data Access logs, costs climbing	Narrow the audit config to only the specific services/permission types you need (e.g. only DATA_WRITE, not DATA_READ), or add an exclusion filter on the sink/_Default bucket for high-volume, low-value log names.
Can't find an event that 'should' be there	Check you're searching the right log type (activity vs data_access vs system_event vs policy) and the right resource container (project vs folder vs organization) — logs for org-level actions don't automatically show up when you're scoped to a single project.
Query in Logs Explorer returns results but the console feels slow	Narrow your time range first, then add filters — Logs Explorer scans much faster with a tight time window.
Need to keep logs beyond the _Default bucket's max retention for compliance	Either raise the _Default bucket's custom retention (up to 3,650 days) or export via a sink to BigQuery/Cloud Storage for indefinite retention outside Cloud Logging's own limits.

9. Best Practices Checklist

- Enable Data Access logs deliberately and narrowly — turn on only the services and permission types (ADMIN_READ / DATA_READ / DATA_WRITE) you actually need for compliance or investigation, to control cost and noise.
- Grant Private Logs Viewer sparingly — it exposes potentially sensitive query text and data-access patterns, not just metadata.
- Export critical audit logs to BigQuery (or Cloud Storage for cold archive) via a sink so retention isn't capped by Cloud Logging's own limits and so you can run rich SQL investigations.
- Build log-based metrics and alerts for the handful of events that really matter — IAM policy changes, service account key creation, firewall rule changes, org policy changes — rather than trying to alert on everything.
- Treat audit logs as sensitive data themselves: they can contain query text, resource names, and IPs. Restrict the exported BigQuery dataset with the same rigor as any other sensitive dataset (consider column-level security / policy tags on the exported table, tying back to the earlier handbook).
- Remember Admin Activity and System Event logs cannot be disabled and are free — there's no reason not to rely on them; the real design decisions are about Data Access and Policy Denied logs.
- For organization-wide visibility, consider an aggregated sink at the org or folder level instead of configuring exports project-by-project.

10. Companion Notebook & Cleanup

A ready-to-run Jupyter/Colab notebook (`audit_logs_demo.ipynb`) accompanies this handbook. It automates Steps 1–5 above using the `google-cloud-logging` and `google-cloud-bigquery` Python client libraries: it creates a demo dataset, enables Data Access audit logs, runs a query, then programmatically reads back the resulting audit log entries and prints the key fields (who / what / when / result) — useful for a live classroom walkthrough without needing to click through the console.

Cleanup

```
# Remove the log sink (stops future exports; existing exported data stays)
gcloud logging sinks delete audit-logs-to-bq --quiet

# Remove the log-based metric
gcloud logging metrics delete iam_policy_changes --quiet

# Remove demo datasets
bq rm -r -f -d ${PROJECT_ID}:audit_demo
bq rm -r -f -d ${PROJECT_ID}:audit_logs_export
```

11. Quick Reference & Further Reading

One-line summary of the whole workflow

```
API call happens → Google Cloud checks IAM permission → an AuditLog entry is writte
n
(type: activity / data_access / system_event / policy) → Log Router applies sinks →
stored in _Required/_Default bucket and/or exported to BigQuery/Storage/Pub-Sub →
you search it via Logs Explorer, gcloud logging read, or SQL on the exported table.
```

Official documentation to bookmark

- Cloud Audit Logs overview — docs.cloud.google.com/logging/docs/audit
- Enable Data Access audit logs — docs.cloud.google.com/logging/docs/audit/configure-data-access
- Types of audit logs (per-service tables) — docs.cloud.google.com/logging/docs/audit#types
- Routing and storage overview (sinks & buckets) — docs.cloud.google.com/logging/docs/routing/overview
- Access Transparency — docs.cloud.google.com/logging/docs/audit/access-transparency-overview

---

End of handbook. Pair this with the companion notebook for the live demo, and the Troubleshooting table in Section 8 during Q&A.;